📍 Waypoint-to-Waypoint Roadnet Navigation

1. Course Content

After completing the topological construction of the road network, the most fundamental and commonly used operation mode is **"precise cruise between specified stations"** (e.g., autonomously driving from Station 1 (loading) to Station 4 (assembly)).

The functionality implemented in this lesson can be intuitively understood as: **giving the car a station number command, and the car automatically searches for the optimal driving route in the topological road network, cruising smoothly along the lane to the designated station**.

---

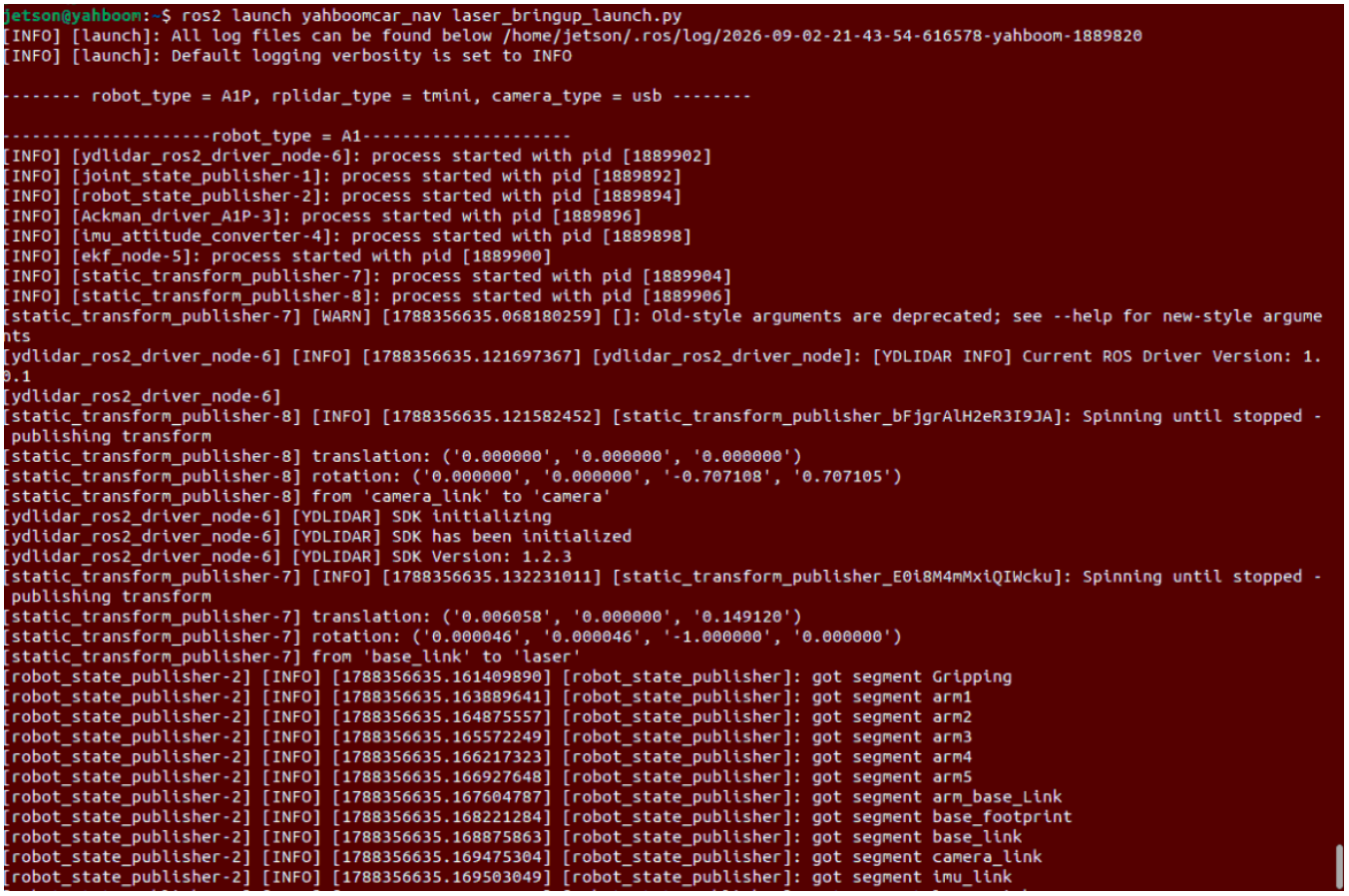
2. Preparation and Running

This case is based on Cartographer high-precision global positioning and the `road_net_route` bridge node. Four terminals need to be opened sequentially for coordinated operation:

2.1 Start Chassis and LiDAR Driver (Terminal 1)

Run on the car host terminal:

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



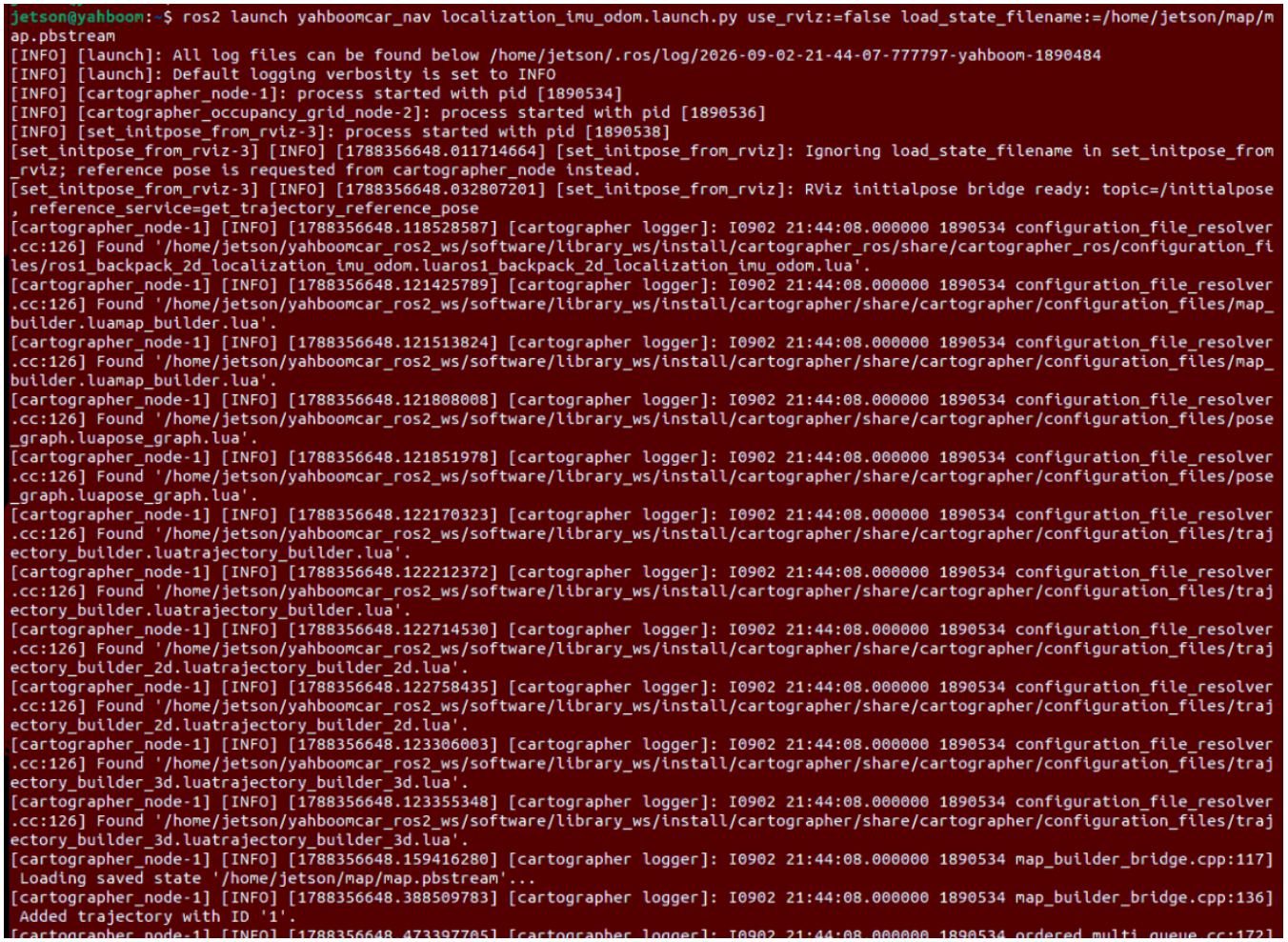
2.2 Start Cartographer Pure Localization Node (Terminal 2)

Use the pose map `map.pbstream` generated in the second lesson for high-precision, fast global relocalization:

```
# RDK X5 system (sunrise user)
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/sunrise/map/map.pbstream

# Or industrial PC jetson user
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/home/jetson/map/map.pbstream

# Raspberry Pi 5 user
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=/root/map/map.pbstream
```



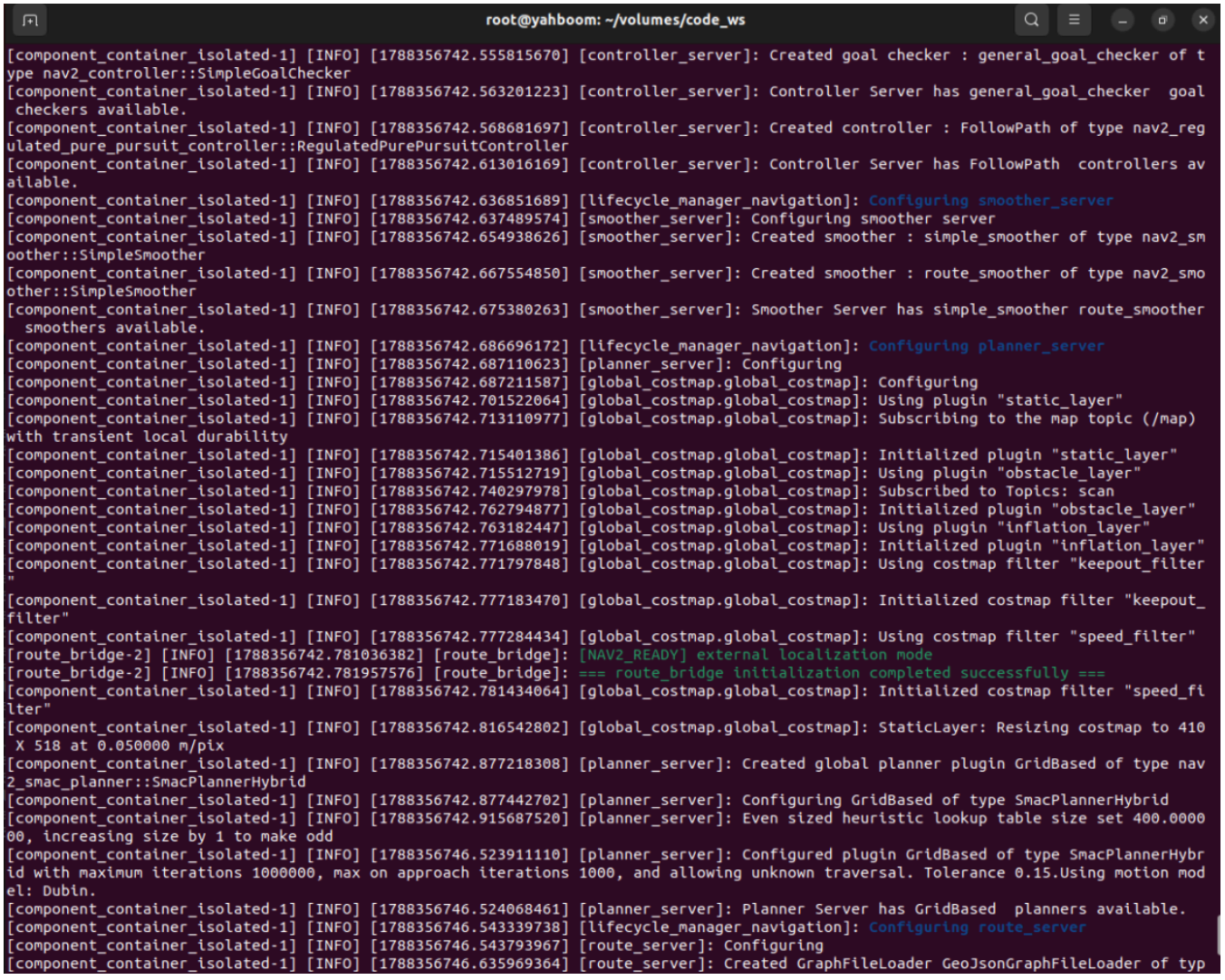
2.3 Start Roadnet Planning Navigation Core Control Node (Terminal 3)

Enter the roadnet docker,

```
# Quick start command
bringup_ros2kilted
# Enter container terminal
exec_ros2kilted
```

This node automatically loads the `map.yaml` grid map, `map.geojson` topological road network, and starts the Nav2 path following controller with the `route_bridge` orchestration engine:

```
ros2 launch road_net_route roadnet_nav2.launch.py use_pose_precise_adjust:=true
map:=map.yaml pose_map:=map roadnet_file:=/root/map/map.geojson
```

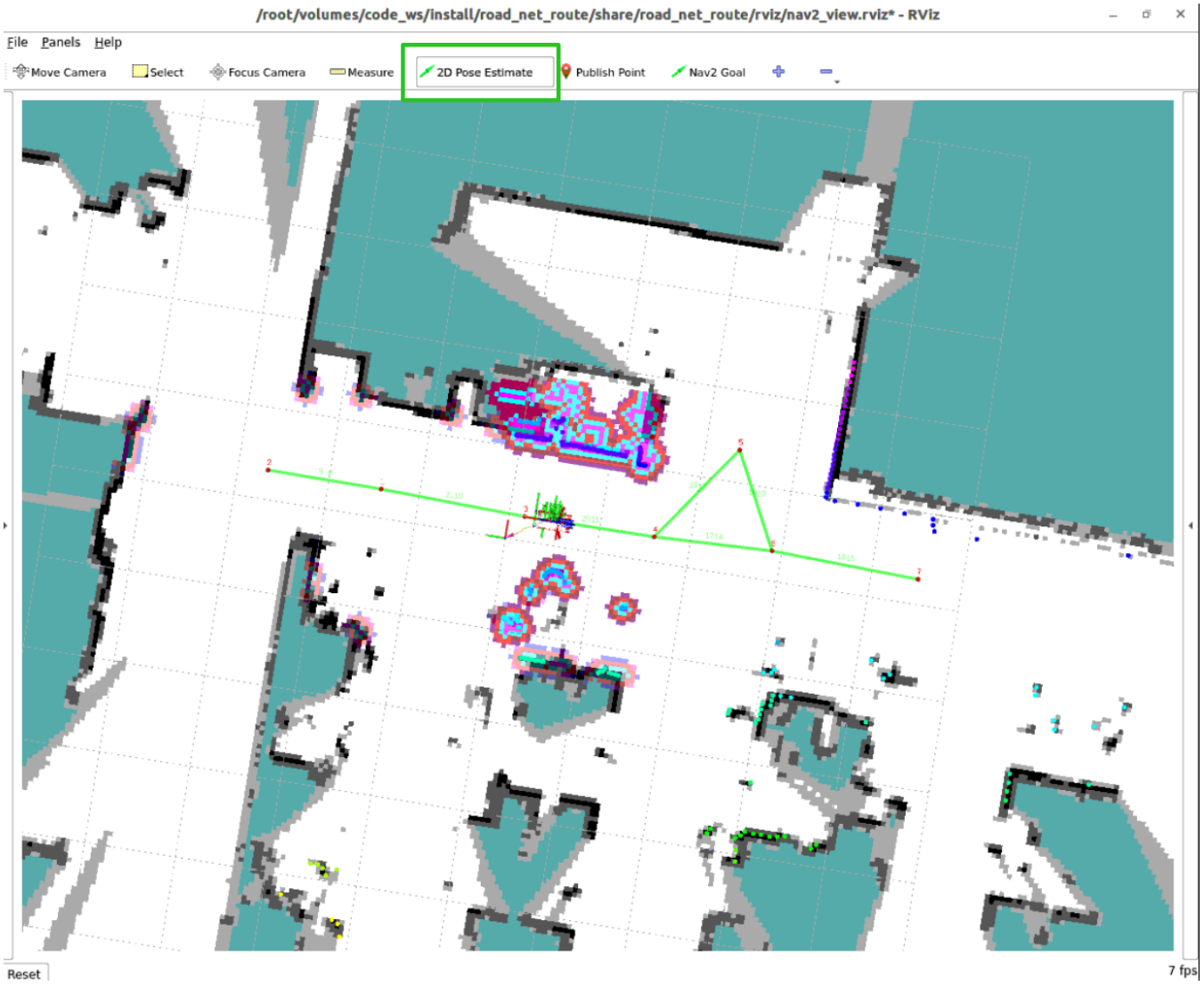


2.4 Start Visualization Display (Terminal 4)

Run the RViz monitoring window on the mainboard roadnet docker side: **(RDK and Raspberry Pi need to run on virtual machine)**

```
ros2 launch road_net_route rviz_only.launch.py
```

If the car's initial pose has not converged, click the **2D Pose Estimate** tool at the top of RViz, long-press and drag on the corresponding map location to assign the initial direction, until the LiDAR point cloud precisely aligns with the black and white obstacle edges on the map:



3. Hands-on Demo and Interaction Flow

3.1 Send Waypoint Navigation Target via Command Line

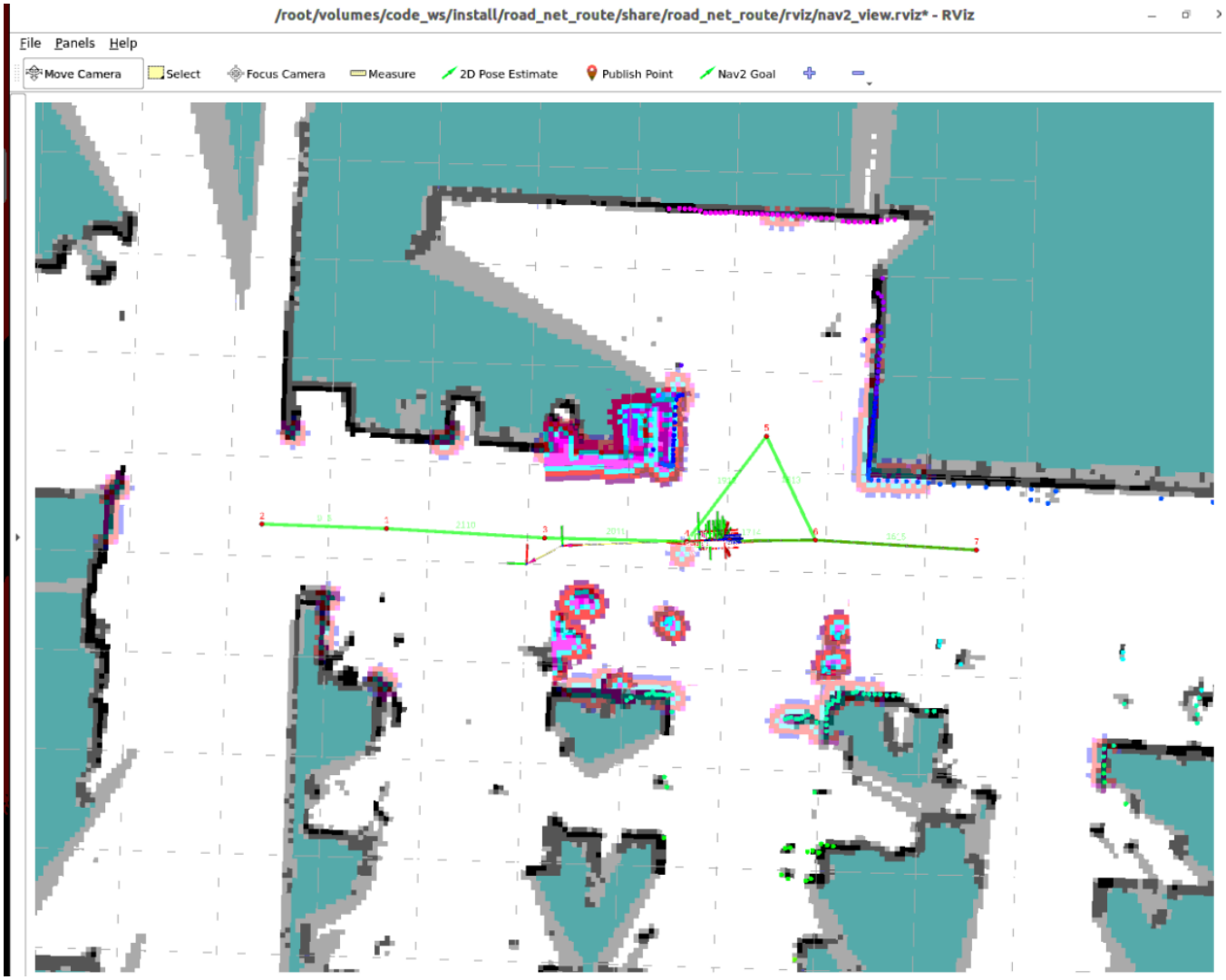
Publishing to the `/road_net_nav_id` topic directly triggers the car's roadnet cruise.

The topic data type is `std_msgs/msg/Int16MultiArray`, with `[start_ID, goal_ID]` passed in the array. For example, cruising from **waypoint 4** (near the car) to **waypoint 7**:

```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [4, 7]}"
```

```
jetson@yahboom:~/map$ ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [4, 7]}"
publisher: beginning loop
publishing #1: std_msgs.msg.Int16MultiArray(layout=std_msgs.msg.MultiArrayLayout(dim=[], data_offset=0), data=[4, 7])
```

The global path planned by the car strictly follows the annotated roadnet edge lines, with smooth transitions at turns, without deviating from the preset road:



3.2 Automatic Bridging Entry Feature When Starting Position is Not on the Road Network

In real working conditions, the robot may have just been powered on or be in a maintenance area, with its position not exactly on any roadnet edge line.

The `road_net_route` system has excellent **adaptive network entry capability (Bridging)**:

1. System detects that the car's current coordinates are not on a roadnet node;
2. Automatically calculates the nearest valid road segment or waypoint to the car's current Euclidean distance in the global road network;
3. Generates a short guided transition trajectory first, smoothly guiding the car onto the main road network;
4. Then seamlessly connects to the topological road network to continue driving toward the target waypoint.

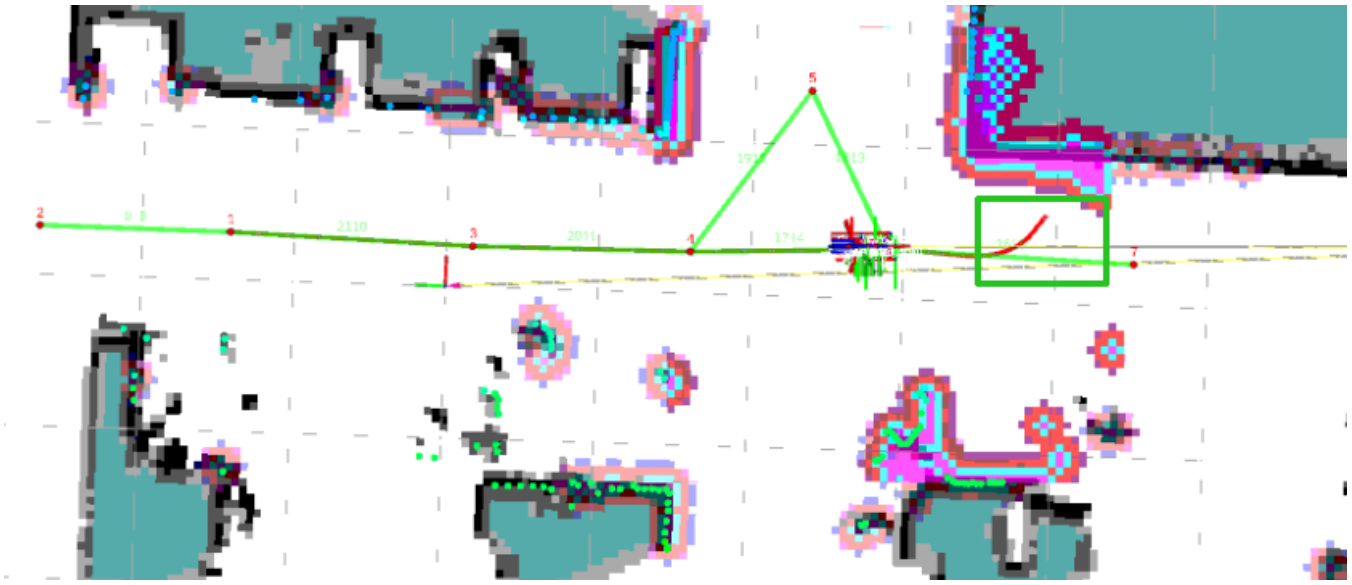
Issue a task from the current nearest point (e.g., 7) to the target point (e.g., 1):

```
ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [7, 1]}"
```

```
jetson@yahboom:~/map$ ros2 topic pub -1 /road_net_nav_id std_msgs/msg/Int16MultiArray "{data: [7, 1]}"
publisher: beginning loop
publishing #1: std_msgs.msg.Int16MultiArray(layout=std_msgs.msg.MultiArrayLayout(dim=[], data_offset=0), data=[7, 1])

jetson@yahboom:~/map$
```

The terminal will print complete path planning details and arrival status feedback:



4. Core Source Code Analysis

Source code reference:

- `code_ws/src/road_net_route/road_net_route/route_bridge.py`
- `code_ws/src/road_net_route/road_net_route/roadnet.py`
- `code_ws/src/road_net_route/road_net_route/path_utils.py`

The entire point-to-point navigation is driven by the `RouteBridge` node, with the core scheduling flow as follows:

```
Receive /road_net_nav_id topic
   ↓
id_nav_callback() validates start and end IDs and pushes into nav_event_queue
   ↓
_handle_nav_events background event thread extracts tasks, calls _navigation(start_id,
goal_id, None)
   ↓
roadnet.shortest_path(start_id, goal_id) runs priority queue Dijkstra to solve optimal
topological waypoint sequence
   ↓
First segment edge direction pre-alignment (Pre-align heading) adjusts car heading to face
lane tangent
   ↓
path_utils constructs continuous path with yaw angles and dense smooth interpolation
(nav_msgs/msg/Path)
   ↓
navigator.followPath(continuous_path) hands over to Nav2 Pure Pursuit controller to drive
chassis tracking
```

```
    ↓
Within goal tolerance range, publish /action_feedback ("road_net_nav_succeeded")
```

4.1 Asynchronous Event Queue and Task Preemption Mechanism

To avoid node hangs caused by executing blocking loops in ROS message subscription callbacks, `route_bridge.py` adopts a **producer-consumer thread architecture**:

```
# route_bridge.py —— id_nav_callback and event queue
def id_nav_callback(self, msg: Int16MultiArray):
    """ID navigation callback: receives start/goal node IDs [start_id, goal_id]"""
    if len(msg.data) < 2:
        self.get_logger().warn("road_net_nav_id requires two integers: start_id and
goal_id")
        return
    self._log.log('info', '\033[95m', 'ROUTE_RX_ID',
                  f"/road_net_nav_id start={msg.data[0]}, goal={msg.data[1]}")
    # Enqueue new task; if old task is executing, automatic cancellation preemption is
triggered
    self._enqueue_navigation_event(msg)

def _handle_nav_events(self):
    """Background dedicated event handling thread"""
    while not self._event_thread_shutdown.is_set():
        event = self.nav_event_queue.get(timeout=0.5)
        if isinstance(event, Int16MultiArray):
            # Unified dispatch to core navigation execution method
            self._navigation(event.data[0], event.data[1], None)
```

4.2 Topological Graph Shortest Path Resolution

In `roadnet.py`, a Dijkstra algorithm optimized with a min-heap priority queue is used to search for the shortest cumulative physical distance node sequence in the topological adjacency table at sub-millisecond speed:

```
# roadnet.py —— shortest_path core algorithm
def shortest_path(self, start_id: int, goal_id: int) -> list[int] | None:
    if start_id not in self.adjacency or goal_id not in self.adjacency:
        return None
    distances = {start_id: 0.0}
    previous = {}
    pq = [(0.0, start_id)]

    while pq:
        current_dist, u = heapq.heappop(pq)
        if u == goal_id:
            # Backtrack to build complete ordered node list [start_id, ..., goal_id]
            path = []
            while u in previous:
                path.append(u)
                u = previous[u]
```

```
            path.append(start_id)
            return list(reversed(path))

        for v, weight in self.adjacency[u].items():
            new_dist = current_dist + weight
            if new_dist < distances.get(v, float('inf')):
                distances[v] = new_dist
                previous[v] = u
                heapq.heappush(pq, (new_dist, v))
    return None
```

4.3 Continuous Dense Trajectory Interpolation and Nav2 Tracking

The Nav2 low-level controller cannot directly execute several discrete sparse waypoint positions. `path_utils.py` performs continuous dense interpolation along the topological road curve at fixed step sizes (e.g., 0.05m or 0.10m), and injects precise orientation quaternions (Heading) for each waypoint:

```
# route_bridge.py —— Continuous trajectory generation and Nav2 invocation
node_path = self.roadnet.shortest_path(start_id, goal_id)

# 1. Expand along edge polylines and interpolate into smooth continuous nav_msgs/msg/Path
continuous_path = path_utils.build_continuous_path_from_nodes(
    self.roadnet, node_path, step=0.05
)

# 2. Publish to /navigation_path for RViz to display global green planning line
self.navigation_path_pub.publish(continuous_path)

# 3. Drive chassis to execute path tracking
follow_task = self.navigator.followPath(continuous_path)
```